

Interacțiunea / Interfețe om-calculator (IOC)

Informatică, anul I · Tehnologia Informației, anul III · Anul universitar 2026–2027, semestrul I

APLICAȚIA-EXEMPLU 09

Temporizator de pauze (Python, Tkinter)

Aceași aplicație desktop în patru variante de calitate: de la o interfață care nu merge la una care merge foarte bine.

Lecția din curs	02 – Factorul uman
Durata	2 ore (Informatică, anul I) · 1 oră (TI, anul III – vezi nota de la final)
Aveți nevoie de	Python 3 (cu Tkinter, inclus în instalarea de pe python.org), Visual Studio Code și folderul <code>ioc-exemple-2026-2027/09-python-temporizator-pauze/</code>
La final aveți	tabelul de observație completat pentru cele patru variante și varianta 2 îmbunătățită până la nivelul 3

1. Obiective

La finalul activității veți putea:

- rula o aplicație Python cu interfață grafică (Tkinter) din VS Code sau din terminal;
- compara patru variante ale aceleiași aplicații și explica, cu argumente, de ce una este mai ușor de folosit decât alta;
- lega problemele observate de factorul uman: atenție, memorie de scurtă durată, model mental, oboseală;
- îmbunătăți o interfață slabă prin modificări mici și precise: etichete, mesaje, stări ale butoanelor, aranjare.

2. Pe scurt: factorul uman și un temporizator de pauze

Lucrul îndelungat la calculator obosește ochii și postura, iar atenția scade în timp. De aceea, ghidurile de ergonomie recomandă pauze scurte și regulate. Aplicația din acest exemplu face un lucru simplu: numără o etapă de lucru (de exemplu, 50 de minute), anunță pauza (5 minute), apoi reia ciclul.

Tocmai pentru că sarcina este simplă, diferențele dintre variante se văd ușor. Utilizatorul are un **model mental** al unui temporizator (pornesc, pun pe pauză, reiau, resetez) și se așteaptă ca aplicația să se comporte la fel [1], [2], [4]. Când interfața îi cere să țină minte coduri („mod 1”, „mod 2”) sau unități nescrise, **încărcarea cognitivă** crește; memoria de scurtă durată este limitată [6]. Când aplicația nu arată ce face, utilizatorul nu poate evalua rezultatul acțiunii sale: este ceea ce Norman numește „prăpastia evaluării” [5].

Cum lucrăm. Exemplul corespunde lecției 02 din cursul dlui Pruteanu (platforma examene.pe.ub.ro).

Variantele sunt notate cu nivelurile propuse la curs: 1 – *nu merge*, 2 – *merge*, 3 – *merge bine*, 4 – *merge foarte bine*.

3. Unde găsiți aplicația și cum o rulați

Unde. În folderul descărcat de pe GitHub (**Code** → **Download ZIP**, apoi dezarhivare): `ioc-exemple-2026-2027/09-python-temporizator-pauze/`. Folderul conține `1-nu-merge.py`, `2-merge.py`, `3-merge-bine.py`, `4-merge-foarte-bine.py` și fișa `FISA.md`.

1. Dacă lucrați pe laptopul propriu, instalați Python 3 de pe python.org. Pe Windows, bifați opțiunea **Add python.exe to PATH**. Tkinter vine odată cu Python; nu instalați nimic cu `pip`.
2. În VS Code: **File** → **Open Folder...** și alegeți folderul `09-python-temporizator-pauze`. Deschideți un terminal: **Terminal** → **New Terminal**.
3. Rulați o variantă (pe Windows, înlocuiți `python` cu `py`):

```
python 3-merge-bine.py
```

Rezultat. Se deschide o fereastră mică, separată de VS Code. Închideți-o înainte să rulați următoarea variantă. (Pe Linux, dacă apare eroarea `No module named 'tkinter'`, instalați pachetul `python3-tk`.)

3.1 Constanta VITEZA (pentru test)

Nimeni nu așteaptă 50 de minute la laborator. La începutul fiecărui fișier există linia de mai jos. Schimbați `1` în `60`, salvați și rulați din nou: un minut din aplicație trece într-o secundă reală, deci etapa de lucru de 50 de minute durează 50 de secunde, iar pauza de 5 minute, 5 secunde.

```
VITEZA = 1 # 1 = timp real; 60 = un minut trece într-o secunda (doar pentru test)
```

Constanta schimbă doar intervalul la care aplicația „bate” o secundă (`1000 // VITEZA` milisecunde); interfața rămâne aceeași. Reveniți apoi la `VITEZA = 1`.

4. Pașii de lucru

1. Rulați cele patru variante **în ordine aleatorie**, nu în ordinea numerelor din nume. Încercați să nu vă uitați la numele fișierului.
2. În fiecare variantă, realizați aceeași sarcină:
 - setați 50 de minute de lucru și 5 minute de pauză și porniți temporizatorul;
 - puneți numărătoarea pe pauză, apoi reluați-o;
 - așteptați finalul etapei de lucru (cu `VITEZA = 60`) și observați cum sunteți anunțați;
 - resetați temporizatorul; încercați și o valoare greșită (de exemplu `abc` sau `-5`).
3. Completați tabelul de observație (secțiunea 5): ce funcționează, ce încurcă, ce principiu este încălcat (de exemplu, una dintre euristicele lui Nielsen [3]) și ce nivel credeți că are varianta.
4. Abia la final apăsați butonul „**Ce nivel are interfața?**” din colțul ferestrei și comparați cu estimarea voastră.

Sfat. Notați lucruri concrete, de exemplu: „după ce apăs Pauza, butonul arată la fel, deci nu știu dacă temporizatorul s-a oprit”. O observație precisă valorează mai mult decât „nu-mi place”.

5. Tabelul de observație

Varianta	Ce funcționează	Ce încurcă	Euristica sau principiul încălcat	Nivelul ghicit
1-nu-merge.py				
2-merge.py				
3-merge-bine.py				
4-merge-foarte-bine.py				



Figura 1. Varianta 2: etichete amestecate, „Reset” lipit de „Start”.

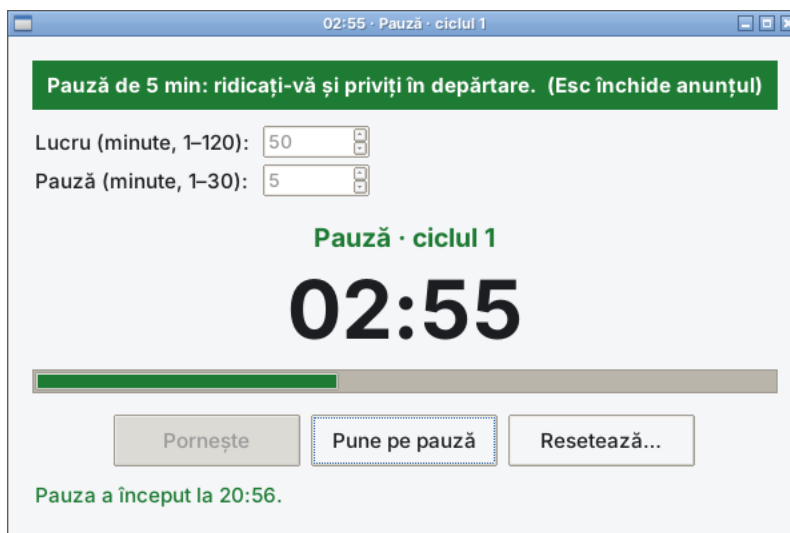


Figura 2. Varianta 4, în pauză: anunț nemodal, etapă și progres colorate, timpul rămas și în titlul ferestrei, butoane active doar când au sens.

6. Exerciții

1. Îmbunătățiți varianta 2 până la nivelul 3. Copiați `2-merge.py` în `2-imbunatatit.py` și realizați:
 - etichete în română, consecvente și cu unitate de măsură („Durata lucrului (minute)”), cu valorile implicite 50 și 5;
 - mesaje care spun ce este greșit și cum se corectează, în locul „Invalid input” și „Error 42”, afișate într-o etichetă de stare;
 - butoane active doar când au sens (`state="disabled"`), iar butonul de pauză își schimbă textul în „Continuă”;
 - mesaje diferite la finalul lucrului și la finalul pauzei, care spun ce urmează;
 - butonul de resetare mutat departe de Start, fără roșu strident, și tooltips pentru butoane;
 - `grid()` cu `columnconfigure(..., weight=1)` în locul coordonatelor fixe din `place()`.
2. ★ (opțional) Refaceți varianta 1 de la zero, păstrând aceeași funcționalitate.
3. ★ (opțional) Adăugați o funcționalitate la varianta 4: programe predefinite (de exemplu 25 / 5, metoda Pomodoro), o temă întunecată sau salvarea setărilor într-un fișier JSON.

Despre proiect. Exemplele din acest repository sunt materiale de lucru: proiectul vostru trebuie să fie original; o variantă copiată sau ușor modificată nu se punctează.

7. Ce arătați la final

- tabelul de observație completat, cu cel puțin două probleme argumentate pentru fiecare variantă;
- fișierul `2-imbunatatit.py` rulând, cu modificările de la exercițiul 1 explicate pe scurt.

Pentru TI anul III (1 oră pe săptămână). Rulați variantele 1 și 3 (cu `VITEZA = 60`), completați tabelul pentru ele și realizați primele trei cerințe de la exercițiul 1. Restul îl puteți termina acasă.

Resurse pentru exersare

- W3Schools – Python (cu exemple „Try it Yourself”): <https://www.w3schools.com/python/>
- Curs de Python pentru începători, în limba română (playlist YouTube): <https://www.youtube.com/playlist?list=PLUVWxwPeA4CxcA2o1ICBgb2elagISVRQ2>
- Documentația Python – tkinter (interfețe grafice): <https://docs.python.org/3/library/tkinter.html>
- Documentația Python – tkinter.ttk (widgeturi cu temă): <https://docs.python.org/3/library/tkinter.ttk.html>
- Nielsen Norman Group – cele 10 euristici de utilizabilitate: <https://www.nngroup.com/articles/ten-usability-heuristics/>

- Cursul IOC al dlui Pruteanu (prezentări, teme, bibliografie): <http://examene.pe.ub.ro/>

Bibliografie

- [1] E. Pruteanu și G. Puiu, *Interacțiunea om-calculator/mașină*. Bacău: Alma Mater, 2009.
- [2] E. Pruteanu, *Factorul uman*, Cursul 2, IOC, Universitatea „Vasile Alecsandri” din Bacău, 2026. Disponibil pe examene.pe.ub.ro.
- [3] J. Nielsen, „10 Usability Heuristics for User Interface Design”, Nielsen Norman Group, 1994, actualizat 2024. nngroup.com
- [4] Y. Rogers, H. Sharp și J. Preece, *Interaction Design: Beyond Human-Computer Interaction*, ed. a 6-a. Hoboken, NJ: Wiley, 2023.
- [5] D. A. Norman, *The Design of Everyday Things*, ed. revizuită. New York: Basic Books, 2013.
- [6] G. A. Miller, „The magical number seven, plus or minus two”, *Psychological Review*, 63(2), 81–97, 1956.
- [7] W3C, *Web Content Accessibility Guidelines (WCAG) 2.2*, 2023. [w3.org/TR/WCAG22](https://www.w3.org/TR/WCAG22)

Ghid pentru cadrul didactic

Citiți această secțiune abia după ce ați completat tabelul.

Nivel	Ce se observă
1 – Nu merge	câmpul „Timp” fără unitate (cere secunde); butoane-simbol (triunghi și pătrat), mici, fără tooltips; pătratul închide aplicația fără confirmare; triunghiul apăsat de două ori pornește două numărători; nu există pauză; secunde brute, gri pe gri; etapa „mod 1 / mod 2”; niciun anunț la final; o valoare greșită este ignorată în tăcere
2 – Merge	etichete amestecate (engleză și română), fără valori implicite; mesaje tehnice („Invalid input”, „Error 42”); Start în timpul numărătorii o reia de la zero, fără avertisment; butonul Pauza nu își schimbă textul; Reset roșu, lângă Start, fără confirmare; „Time is up!” identic la ambele etape, într-o fereastră care oprește numărătoarea; poziții fixe
3 – Merge bine	ttk, etichete cu unități, valori implicite și limite; etapă colorată, timp mare, bară de progres, numărul ciclului; butoane active doar când au sens; mesaje de stare și tooltips; alertă clară cu indicații de ergonomie; fereastră redimensionabilă
4 – Merge foarte bine	câmpuri care acceptă doar cifre; confirmare la resetare, cu „Nu” implicit; anunț nemodal (banner, sunet, fereastra în față); timpul rămas în titlul ferestrei; focus gestionat, Enter și Spațiu acționează butonul, scurtături Ctrl+R și Esc; contrast verificat (WCAG AA), inel de focus colorat